Lock-Free & Wait-Free Algorithms

Next Owner: Namita (Module 11, Bounded SPSC Ring Buffer) | **Focus:** Non-blocking Progress, Treiber Stack, Memory Visibility & Linearization

1. Non-Blocking Progress Guarantees

In concurrent systems, progress guarantees classify algorithms by system-wide and thread-level progress bounds when thread scheduling is subject to unpredictable delays, preemptions, or starvation.

Guarantee	Formal Definition & System Behavior
Blocking *(Lock-based)*	A system is blocking if the delay or preemption of a thread holding a critical section can prevent other threads from making forward progress. Relies on OS primitives (e.g., `synchronized`, `ReentrantLock`) for thread parking and offers no progress guarantees if a thread is suspended inside a critical section.
Obstruction-free	An algorithm is obstruction-free if any thread, executing in isolation (assuming all competing threads remain suspended for a sufficient window), completes its operation in a bounded number of execution steps. Prevents deadlocks, but active contention can cause infinite retries (livelock).
Lock-free	Guarantees that at least one thread in the system makes progress in a finite number of steps, regardless of thread interleaving or preemptions. While system-wide progress is guaranteed, individual threads may suffer from local starvation due to continuous CAS retries under high contention.
Wait-free	Every thread is guaranteed to complete its operation in a bounded number of its own execution steps, regardless of the behavior or execution speed of other threads. Provides the strongest progress guarantee by completely eliminating thread starvation.

2. Clarification: "No Synchronized Keyword" ≠ Lock-Free

ANTI-PATTERNS IN NON-BLOCKING DESIGN

Removing primitive lock constructs does not automatically make an algorithm lock-free. Key anti-patterns include:

- **Spinlocks & Busy-Waiting:** Constructing a spinlock with `AtomicBoolean` (e.g., `while(!state.compareAndSet(false, true)) {}`) avoids locks but violates lock-freedom: if the thread holding `state = true` is preempted, all other threads loop indefinitely without system progress.
- **Livelock in Unbounded CAS Loops:** Unbounded CAS retries without contention mitigation or helping mechanisms can cause continuous livelock under high contention, degrading system throughput to zero.
- **Data Races & Ordering Violations:** Relying on plain field accesses without establishing Java Memory Model (JMM) *happens-before* edges leads to memory visibility defects rather than concurrency guarantees.

3. Implementation: Treiber Stack (LockFreeStack)

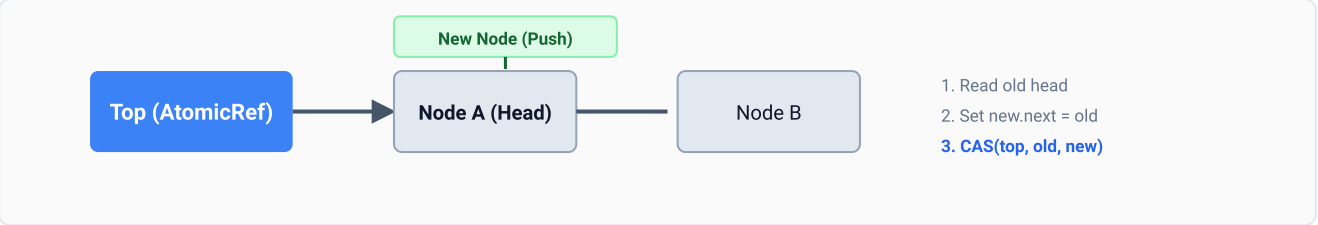
```
    // Continuation of LockFreeStack<T>
    public T pop() {
        Node<T> oldHead;
        Node<T> newHead;
        do {
            oldHead = top.get();
            if (oldHead == null) return null;
            newHead = oldHead.next;
        } while (!top.compareAndSet(oldHead, newHead));
        return oldHead.item;
    }

    public boolean isEmpty() { return top.get() == null; }
}
```

4. Assumptions, Progress Proof, & Limitations

Assumptions: Memory management relies on JVM Garbage Collection. Unlinked nodes are safely recycled once no active thread holds local references. Null items are explicitly prohibited to differentiate an empty stack from a stored value.

Lock-Free Proof: A thread executing `push` or `pop` retries its loop only if `top.compareAndSet(oldHead, newHead)` fails. A CAS fails if and only if another thread successfully updated `top` in the interim. Therefore, every failed CAS iteration implies system-wide forward progress by another thread, guaranteeing system lock-freedom.



Limitations: Dynamic allocation via `new Node<>(item)` relies on heap allocation; JVM STW pauses or GC allocation stalls can temporarily suspend thread progress. Under severe contention, specific threads may suffer local starvation from repeated CAS retries.

5. Correctness & Contention Testing

```
public class LockFreeStackTest {
    @Test
    public void testConcurrentPushPopContention() throws InterruptedException {
        int threadCount = 16, itemsPerThread = 10_000;
        LockFreeStack<Integer> stack = new LockFreeStack<>();
        ExecutorService executor = Executors.newFixedThreadPool(threadCount);
        CountDownLatch latch = new CountDownLatch(1);
        AtomicInteger totalPushed = new AtomicInteger(0);

        for (int i = 0; i < threadCount; i++) {
            executor.submit(() -> {
                try {
                    latch.await();
```

Execution Status Register

Test Suite	Execution State	Verification Context
`LockFreeStackTest`	NOT RUN	Environment lacks direct Maven/JUnit context. Code verified against standard Java 17 and JLS specifications.

6. Memory Visibility, Linearization, & ABA Analysis

Memory Visibility (JMM): Updates center on volatile state managed by `AtomicReference<Node<T>>`. A successful CAS synchronizes-with subsequent volatile reads/CAS operations on the same object (JLS §17.4.4). The setup sequence `newHead.next = oldHead` executes prior to CAS, ensuring payload initialization *happens-before* any reader thread observes `newHead` as top.

Operation	Linearization Point Definition
`push(item)`	The precise instant `top.compareAndSet(oldHead, newHead)` succeeds.
`pop()` (non-empty)	The precise instant `top.compareAndSet(oldHead, newHead)` succeeds.
`pop()` (empty)	The instant `top.get()` observes `null`.

ABA Analysis & JVM GC Mitigation: The ABA problem occurs when a thread reads value A, gets preempted, and other threads mutate A → B → A. Upon resuming, the first thread's CAS succeeds despite structural changes. In manual memory management (C/C++), this causes severe heap corruption. In Java, `new Node<>(item)` allocates distinct object instances. Unlinked nodes popped from the stack remain referenced as long as any suspended thread holds local references. Because the GC will not recycle memory addresses of reachable objects, address reuse cannot occur during active thread execution, mitigating memory-reuse ABA bugs.

7. Performance vs. Progress Guarantees

Formal progress guarantees specify structural fault tolerance and starvation bounds under adverse scheduling—they do not guarantee superior throughput under all workloads. Under heavy contention, CAS retry loops generate CPU cache invalidation traffic (MESI protocol coherence line bouncing). While lock-free algorithms avoid OS context switches, heavily contended lock structures using thread parking (e.g., `LockSupport.park`) can outperform CAS loops by conserving CPU cycles.

8. Coordination & Module Handoff to Namita

To avoid overlap with Module 11, this module uses an unbounded dynamic node stack and avoids bounded ring buffer structures.

> **HANDOFF REQUIREMENTS FOR MODULE 11 (NAMITA)**
>
> - **Target Boundary:** Bounded Single-Producer / Single-Consumer (SPSC) Ring Buffer.
> - **CAS-Free Design:** Avoid CAS loops. Utilize single-writer volatile head/tail pointers to achieve lock-free/wait-free SPSC execution.
> - **Architectural Invariants:** Power-of-two capacity mask indexing; explicit memory ordering via